

# NeuroPay Progress Report

A line-by-line comparison of the AI-Powered Centralized Subscription Management System as proposed against what exists in the `frontend-dev` branch today — plus a feasibility-checked feature roadmap to close the gap.

| PROJECT                                                                     | TEAM                                                              | SUPERVISOR       | PROPOSAL DATED |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------|------------------|----------------|
| AI-Powered Centralized Subscription Management System for Banking Platforms | Wania Khan · Shahzaib Sajjad · Esha Arshad                        | Miss Huma Rashid | 12 Mar 2026    |
| REVIEW DATE                                                                 | REPO STATE REVIEWED                                               |                  |                |
| 27 Aug 2026                                                                 | branch <code>frontend-dev</code> ,<br>commit <code>6ceb1c1</code> |                  |                |

01 — EXECUTIVE SUMMARY

# One layer is real. The layer the FYP is graded on isn't built yet.

The repository currently contains a fully designed React Native (Expo) mobile shell and a Spring Boot authentication service — both genuinely solid. What it does not yet contain is any part of the system the proposal's four research objectives actually promise: a subscription data model, a transaction pipeline, an AI detection engine, or fraud/anomaly logic. Every subscription, alert, and "AI insight" visible in the app today is a hard-coded array in `frontend/constants/mockData.ts`, not a live signal from the backend.

1 / 4

Proposal objectives with backend code behind them (auth only)

0

Lines of AI/ML code in the repository

2

Database tables exist — `users`, `password_reset_tokens`

15+

Frontend screens fully built, mostly wired to mock data

**Schedule check against your own Gantt chart**

The proposal is dated 12 March 2026; today is roughly week 24 of the stated 43-week plan (approximate, read off the submitted chart). By this point, Phase 2 (weeks 13–22) commits the team to a working *Transaction & Subscription Service*, an *Initial Subscription Detection Model*, and a *Fraud Detection Algorithm Prototype* — none of which exist in the repo. What *has* been delivered ahead of schedule is *Security & Authentication Modules*, formally a Phase 3 (week ~28) item, plus a level of frontend dashboard polish beyond what Phase 2 asked for. In short: the team has been building the shell and the login system very well, while the graded core — subscription intelligence — hasn't started. That's the single most important finding in this report.

The rest of this document breaks that gap down feature-by-feature, separates what's genuinely done from what only *looks* done in the UI, and lays out a feasibility-checked path — including several commercially interesting additions (a subscription marketplace, plan comparison, AI cost recommendations) — to bring the build back in line with the proposal before final submission.

# Every proposed capability, checked against the code

Status reflects what runs today, not what's designed on screen. "Partial" means either the UI exists with no backend, or the backend exists with no real logic behind it.

| PROPOSED CAPABILITY                              | STATUS                   | EVIDENCE IN REPO                                                                                                                                                          |
|--------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Centralized subscription dashboard               | ● Partial — UI only      | <code>app/(tabs)/index.tsx</code> , <code>subscriptions.tsx</code> render <code>constants/mockData.ts</code> ; no subscription API                                        |
| AI subscription detection (recurring-payment ML) | ● Not implemented        | No Python/ML code anywhere in the repository; no transaction data to detect patterns from                                                                                 |
| Fraud alerts (price hikes, suspicious charges)   | ● Not implemented        | <code>app/(tabs)/alerts.tsx</code> renders 4 hand-written alert objects; no detection engine, no alert table                                                              |
| Inactive-subscription monitoring                 | ● Not implemented        | No usage-tracking field, job, or model anywhere; "haven't used Figma in 45 days" is a hard-coded string                                                                   |
| User controls — approve / pause / block          | ● Partial — display only | Status badges render in <code>subscriptions.tsx</code> ; no endpoint changes a subscription's state                                                                       |
| Notifications (app / email / SMS)                | ● Partial                | <code>MailService.java</code> sends welcome & reset-password email only; no subscription/fraud channel, no SMS, no push                                                   |
| Auth & account security                          | ● Complete               | <code>AuthController</code> , <code>JwtService</code> , <code>SecurityConfig</code> — signup/signin/forgot/reset, fully wired end-to-end                                  |
| Virtual card issuance                            | ● Partial — cosmetic     | <code>VirtualCardService.java</code> generates a unique-looking 16-digit number; no card network, no PCI scope, no linked ledger                                          |
| Wallet balance / funding                         | ● Partial — fragile      | Balance lives only in browser <code>localStorage</code> ( <code>AuthContext.tsx</code> ); no-ops on native iOS/Android, so it doesn't persist on an actual mobile install |
| External funding cards (add/edit/remove)         | ● Partial — UI only      | <code>cards/index.tsx</code> keeps cards in local component state seeded from <code>initialExternalCards</code> ; resets on reload                                        |

03 — WHAT'S ACTUALLY COMPLETED

Solid, production-shaped work — mostly in one layer

This is real, working code, not a mockup. It's just concentrated in identity/auth and the visual shell rather than the subscription domain.

Backend — authentication service

- Full JWT sign-up / sign-in with BCrypt password hashing, access + refresh tokens ( `JwtService` , `JwtAuthFilter` , `SecurityConfig` )
- Forgot-password / reset-password flow with a real expiring-token entity ( `PasswordResetToken` , `PasswordResetTokenRepository` ) rather than a stubbed endpoint
- Transactional HTML email via Gmail SMTP + Thymeleaf templates for welcome and reset-password mail ( `MailService.java` , `MailConfig.java` ), sent asynchronously
- Consistent API envelope ( `ApiResponse<T>` ) and a global exception handler covering duplicate-user, invalid-token, and not-found cases
- Role enum and Spring Security wiring in place for future authorization checks

Frontend — application shell

- Complete Expo Router navigation: auth stack (login, signup, forgot/reset password, add-card) + 5-tab shell (home, subscriptions, wallet, alerts, profile) + a cards stack
- Auth context wired live to the backend ( `services/api.ts` , `context/AuthContext.tsx` ) — sign-up/sign-in genuinely round-trip to Spring Boot and store the JWT
- A polished, consistent dark-mode design system (Manrope type scale, motion via Reanimated) applied across every screen
- A believable virtual-card visualization component ( `components/ui/CreditCard.tsx` ) reflecting the real backend-issued card number

04 — PARTIALLY COMPLETED

Looks finished on screen; the other half is missing

These are the highest-leverage fixes: the UI work is already paid for, so closing each gap is mostly backend and wiring, not redesign.

Wallet balance

● Needs backend

Deposit/freeze UI is complete and feels real, but the balance is a browser-only `localStorage` number with no server-side wallet entity or transaction ledger. On native Android/iOS it silently doesn't persist at all — `safeStorage` is a web-only no-op.

Virtual card issuance

● Cosmetic

`VirtualCardService` guarantees a unique-looking number, but it isn't attached to a real card network, has no spend limit enforcement, and can't actually authorize a charge.

External funding cards

● UI only

Add/edit/delete all work against in-memory component state seeded with two demo cards; nothing survives a reload, and there's no encryption/tokenization story for card data at all.

Notifications

● Auth-only

The email pipe is real and reusable — it just isn't hooked to anything except auth. There's no notification table, no in-app inbox backing the Alerts tab, and no SMS/push transport.

Profile screen

● Not wired

`profile.tsx` doesn't call `useAuth()` at all — name, email, and the "\$247/mo · \$540 saved" stats are hard-coded, so it currently shows the same fake profile for every logged-in user.

## The core of the FYP is currently a blank slate

These are the objectives the proposal is actually graded against. None have any code, schema, or scaffolding yet.

- **AI microservice** — the proposal's own architecture calls for a Python/scikit-learn/TensorFlow service; zero Python files exist in the repository.
- **Subscription & Transaction data model** — no entities, repositories, or migrations for subscriptions, transactions, or merchants. There is nothing yet for an AI model to detect patterns *from*.
- **Recurring-payment / subscription detection logic** — rule-based or ML — does not exist.
- **Fraud / anomaly scoring with confidence output** — does not exist.
- **Inactive-subscription usage tracking** — no usage signal is ever captured, so "unused for 45 days" can't be computed.
- **Subscription lifecycle endpoints** — approve, pause, block, cancel — none are exposed by the backend.
- **Notification service as its own component** — the architecture diagram calls for one; today it's two email templates bolted onto auth.
- **Automated tests** — beyond the default Spring Boot scaffold test ( `NeuroPayApplicationTests` ), there is no test coverage on either side of the stack.

# What would make this commercially credible, not just complete

Grouped by theme. Each item carries a feasibility swatch — see the matrix in the next section for what the swatches mean.

## A. Subscription marketplace & comparison

Netflix, Spotify, ChatGPT/Gemini, cloud storage, gym — browse and compare in-app

- In-app plan catalog**

Possible now

A curated, admin-maintained list of common subscriptions with tiers, price, and features — Netflix Basic/Standard/Premium, Spotify, ChatGPT Plus/Pro, iCloud+/Google One, gym memberships.
- Side-by-side plan comparison**

Possible now

Compare two or more catalog entries (or a catalog entry against the user's current active subscription) on price, features, and billing cycle.
- One-tap "subscribe" via the virtual card**

Possible now

Debit the in-app wallet ledger and record a new subscription — a sandboxed checkout, not a real charge to Netflix/OpenAI.
- Actually creating the external account**

Not realistically possible

Genuinely provisioning a real Netflix, Spotify, or ChatGPT subscription on the user's behalf.

## B. AI & intelligence

The part the grade actually hinges on

- Rule-based recurring-payment detector (v1)**

Possible now

Group transactions by merchant + amount tolerance + interval (7/30/365 days) before reaching for ML — a believable, gradeable "detection" step that also produces labelled data for a later model.
- ML anomaly / fraud scoring**

Possible now, on synthetic data

Isolation Forest or a small autoencoder over transaction features, trained on synthetic/sandbox data, producing the confidence score the proposal promises.
- Merchant name normalization & auto-categorization**

Possible now

Turn "NFLX\*8817262" into "Netflix / Entertainment" via a small NER model or an LLM call, then group spend by category.
- Personalized savings recommendations**

Possible now

"You haven't opened Figma in 45 days — pausing saves \$180/yr" generated from real usage/billing data rather than typed as a string.
- Bank-grade fraud accuracy at the 95%+ figure quoted in the pitch deck**

Significant R&D — treat as a stretch target, report honestly against synthetic-data accuracy

Requires volumes of real labelled transaction data no student project has access to.

## C. Payments & banking

- Real wallet ledger**

Possible now

A proper double-entry transaction table in Postgres backing balance, deposits, and subscription debits — replacing the `localStorage` number.

Real top-ups via a payment gateway

Needs a gateway account

Stripe (or a Pakistan-market PSP like JazzCash/Easypaisa) sandbox for genuinely funding the wallet from a real card.

Multi-bank aggregation (Plaid/TrueLayer-style)

Possible internationally with a partner; not realistically available in Pakistan today — there is no public open-banking API layer to integrate with

Reading real transactions across a user's actual bank accounts, as the pitch deck's "direct banking integration" implies.

A real bank's fraud webhook / core-banking integration

Not realistically possible for a self-issued FYP card

What "banking platform" integration would mean in production.

D. Notifications & engagement

Push notifications

Possible now

Expo's push service, already available for free inside the current Expo project.

In-app notification center

Possible now

A real inbox with read/unread state backing the Alerts tab, replacing the 4 mock alert objects.

SMS alerts

Needs a Twilio-class account

Twilio (or a local aggregator) for the "SMS" channel promised in the proposal.

Biometric login

Possible now

expo-local-authentication — Face ID / fingerprint unlock, matching the pitch deck's roadmap slide.

E. Analytics & controls

Category-wise spend breakdown & monthly trend

Possible now

Once real transaction data exists, this is a straightforward chart layer on top of it.

Spend forecasting

Possible now

Simple linear/seasonal projection of next month's subscription spend based on active plans.

Budget caps with auto-block

Possible now

Let a user cap monthly subscription spend and have new charges above the cap auto-flag or auto-pause.

Family / shared subscription splitting

Split one subscription's cost across linked household accounts.

Possible now

07 — FEASIBILITY MATRIX

# What's realistic to promise in the final report

Ordered by dependency, not difficulty of code alone — a "possible now" item still takes real engineering effort, it just doesn't wait on anyone outside the team.

- Possible now, in-house

Possible — needs a 3rd-party API / account

Requires significant R&D or data the team doesn't have
- Not realistically possible at FYP scope

| FEATURE                                                               | FEASIBILITY                           | WHY                                                                       |
|-----------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------|
| Rule-based subscription detection                                     | <div>Possible now</div>               | Pure logic over your own transaction table                                |
| In-app marketplace & plan comparison                                  | <div>Possible now</div>               | Static/admin-curated catalog data, no external dependency                 |
| Wallet ledger, freeze/unfreeze, spend caps                            | <div>Possible now</div>               | Standard backend CRUD + business rules                                    |
| ML anomaly/fraud scoring (synthetic data)                             | <div>Possible now</div>               | Off-the-shelf scikit-learn models on a generated dataset                  |
| Push notifications, biometric login                                   | <div>Possible now</div>               | Already available inside the current Expo SDK                             |
| Real payment gateway top-ups                                          | <div>Needs a partner</div>            | Requires a Stripe/JazzCash/Easypaisa merchant account                     |
| SMS delivery                                                          | <div>Needs a partner</div>            | Requires Twilio or a local SMS gateway account                            |
| Multi-bank account aggregation (global markets)                       | <div>Needs a partner</div>            | Plaid/TrueLayer-class provider, outside Pakistan                          |
| 95%+ accuracy fraud detection on real bank data                       | <div>Significant R&amp;D</div>        | Needs volume/variety of labelled real transactions no student project has |
| Automated real-world subscription cancellation on the provider's side | <div>Significant R&amp;D</div>        | No standard API most subscription merchants expose for this               |
| Real multi-bank Open Banking integration in Pakistan                  | <div>Not realistically possible</div> | No public open-banking API layer exists in the Pakistani market today     |
| Genuinely provisioning real Netflix/Spotify/ChatGPT accounts          | <div>Not realistically possible</div> | No merchant-side self-serve provisioning API exists for this              |
| Direct core-banking / real fraud webhook integration                  | <div>Not realistically possible</div> | Requires being an actual regulated bank, not a self-issued virtual card   |



## What's left to build, itemized

### ■ Frontend

- Replace every mock array in `mockData.ts` with live API calls
- Wire wallet screen to real backend ledger endpoints
- Build a real notification center + push registration
- Add approve/pause/block actions calling the backend
- Add biometric login option
- Build marketplace & plan-comparison screens
- Wire cards screen to real funding-card CRUD
- Wire the profile screen to `useAuth()` and real stats
- Add spend analytics charts

### ■ Backend

- Design & implement Subscription, Transaction, Merchant, Alert, Notification, Wallet entities
- Subscription CRUD + status-change (pause/block/approve) endpoints
- Wallet service — balance, deposit, ledger, server-side freeze state
- Role-based authorization beyond the current `Role` enum
- Real unit/integration test suite
- Transaction ingestion endpoint/service (simulated bank feed is acceptable)
- Alerts/notification REST + delivery service, extending `MailService`
- Marketplace/catalog endpoints + sandboxed checkout
- Move DB password, mail app password, JWT secret out of `application.yml`

### ■ AI / ML

- Stand up the Python microservice (FastAPI/Flask) per the proposed architecture
- Recurring-payment detector — start rule-based, evolve toward clustering
- Merchant categorization (NLP or LLM-assisted)
- Training/evaluation harness with an honestly reported accuracy figure
- Data pipeline: ingest & normalize transaction data with pandas
- Anomaly/fraud scoring model with a confidence output
- Inactive-subscription usage inference
- Real-time inference endpoint the Spring Boot backend can call

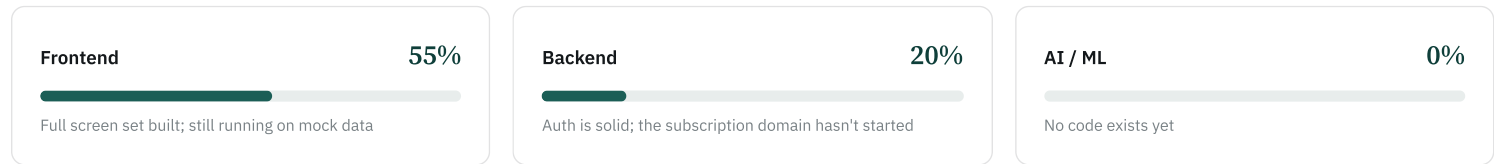
### ■ Third-party integrations

- Payment gateway sandbox (Stripe / JazzCash / Easypaisa)
- SMS (Twilio or local equivalent)
- Optional: an LLM API (Claude/GPT) for merchant normalization and natural-language spend insights
- Push notifications (Expo Push / FCM)
- A realistic synthetic transaction generator standing in for a bank feed — clearly labelled as simulated in the final report

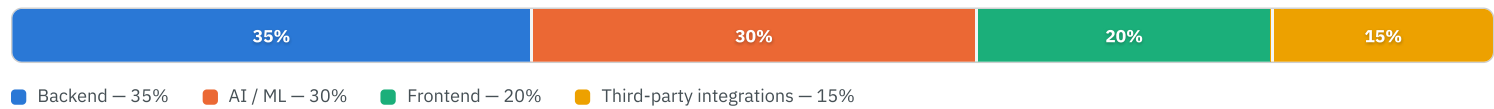
# Where the project stands, and where the remaining effort sits

Qualitative sizing based on scope breadth of the checklists above, not timed estimates — useful for prioritizing, not for billing.

## Current completion, by layer



## Where the remaining effort lies



Backend and AI dominate what's left because they're starting from zero on the domain that matters most for grading, while the frontend only needs re-wiring rather than redesign.

# Five phases to close the gap before final submission

Sequenced so each phase produces something demoable, and so the AI model has real data to train on by the time it's built.

1

## Core data model & transaction pipeline

WEEKS 1–2

Stand up the domain the rest of the system depends on.

- Subscription, Transaction, Merchant, Wallet entities + migrations
- A synthetic/sandbox transaction generator, clearly labelled as simulated
- Wallet ledger replacing the `localStorage` balance

2

## Rule-based subscription & fraud detection (v1)

WEEKS 3–4

A believable, working detection layer before reaching for ML — and the source of labelled data for it.

- Recurring-payment detector (merchant + amount tolerance + interval)
- Price-increase and unknown-merchant flags
- Real alerts backing the Alerts tab, replacing the mock array

3

## AI microservice

WEEKS 5–7

The proposal's centerpiece — built on data the earlier phases now produce.

- Python/FastAPI service; scikit-learn anomaly model with a confidence score
- Merchant categorization
- Honest accuracy evaluation reported against the synthetic dataset

4

## User controls & notifications

WEEKS 8–9

Close the loop the proposal promises: users acting on what the system detects.

- Approve / pause / block endpoints wired to the frontend
- Push notifications; notification center screen
- Budget caps with auto-flagging

5

## Marketplace, analytics & demo hardening

WEEKS 10–12

The commercial-credibility layer, plus getting the whole thing demo-ready.

- Plan catalog + comparison + sandboxed one-tap subscribe
- Category spend analytics & forecasting
- Secrets moved out of `application.yml`; core test coverage; final QA pass

## Fix these regardless of feature priority

### Secrets are committed in plaintext

`application.yml` contains a live-looking Postgres password, a Gmail app password, and the JWT signing key, all checked into the repository. Rotate these credentials and move them to environment variables before the repo is ever shared publicly or with markers outside the team.

### Wallet balance doesn't survive a real mobile install

`safeStorage` in `AuthContext.tsx` only reads/writes on `Platform.OS === "web"`. On an actual Android/iOS build, deposits silently vanish on next launch — worth fixing before a live demo on a phone, not just in the browser preview.

### Profile screen shows fake data to every user

`profile.tsx` hard-codes a name, email, and spend stats instead of reading `useAuth()` — a quick, high-visibility fix.

### Calibrate the "banking integration" claim

The pitch deck's "direct banking integration" and "Pakistan Focus... local banking APIs" language over-promises relative to what's achievable without an actual bank partner. Recommend reframing the final report around a *self-issued virtual card + simulated bank feed*, which is exactly what's buildable, rather than implying live third-party bank connectivity.

# The five moves that matter most before submission

## 1 Build the data model first, not the model

A  
Subscription/Transaction  
schema  
and  
a  
synthetic  
feed  
unlock  
everything  
else  
—  
detection,  
alerts,  
and  
analytics  
all  
read  
from  
it.

## 2 Ship rule-based detection before ML

It's  
gradeable  
now,  
produces  
the  
labelled  
data  
an  
ML  
model  
needs  
later,  
and  
de-  
risks  
the  
AI  
phase  
if  
time  
runs  
short.

## 3 Re-wire the frontend instead of rebuilding it

The  
UI is  
already  
at  
production  
quality  
—  
replacing  
`mockData.ts`  
with

real  
API  
calls  
is  
the  
fastest  
path  
to a  
genuinely  
working  
demo.

4     Add the marketplace as the differentiator

Plan  
comparison  
and  
sandboxed  
subscribe/cancel  
are  
fully  
achievable  
in-  
house  
and  
are  
exactly  
the  
kind  
of  
feature  
that  
reads  
as  
commercially  
credible  
to a  
panel.

5     Rotate the committed secrets and fix native persistence

Both  
are  
quick,  
both  
are  
the  
kind  
of  
thing  
a  
careful  
reviewer  
will  
notice  
immediately.